```
data-sort-value="<%= favorite.sort_date %>"
data-controller="css"
data-css-target="elementToChange"
data-css-css-class="animate__slideInRight"
data-css-status-value="<%= animate_in ? "true" : "false" %>"
data-action="animationend->css#toggle">
```

This code adds the CssController to the same article element with a few data attributes. The data-controller=css attribute establishes that the controller is there, and we also set the element up as its own target with data-css-target="elementToChange". The next attribute, data-css-css-class="animate__slideInRight" says that the CSS controller cares about whether that animation class is in or out of the class list based on the status value. The status value is set with data-css-status-value="<%= animate_in ? "true" : "false" %>"—in other words, the starting status is true if we want the element to animate; otherwise, not. Finally, the data-action="animationend->css#toggle" sets a different data action. Our past uses of the CssController have been toggled by click; this time, we're toggling when the animation ends.

If we start this partial with animate_in set to true, then the data-css-status-value is true. When Stimulus connects the CSS controller, it calls statusValueChanged, and since the status value is true, it adds the CSS class, animate__slideInRight to the class list. The element then animates in. When the animation ends, the animationend event is fired, causing Stimulus to call toggle on the controller, setting the status value to false, and removing the animate__slideInRight class. Now if you add a second favorite, the existing class no longer has an animation CSS attached and doesn't animate again.

I realize that you might look at all this HTML and think it's verbose. All I can say is it made me genuinely happy, in real time, as I wrote this—this was not a planned digression, but a bug I found while writing the example and fixed quickly without writing any new JavaScript.